RAPPORT DU PROJET S3

Conception et Implémentation d'une Base de Données

Auteurs : Rharrabti Yassine, Bouzid Youcef, Najarzadeh Banadaki Roham

Groupe : 3A

Date : 10/12/2025

Table des matières

RAPPORT DU PROJET S3.....1
Conception et Implémentation d'une Base de Données.....1
1. Introduction.....3
2. Le Modèle Relationnel Commenté.....3
2.1. Schéma Relationnel.....3
2.2. Justification des choix d'implémentation.....4
3. Implémentation Statique et Jeu de Données.....5
3.1. Stratégie de peuplement.....5
4. Exploitation : Vues et Requêtes.....5
4.1. Vues Pédagogiques.....5
4.2. Vues de Contrôle et Audit.....5
4.3. Vues Statistiques.....5
5. Implémentation Dynamique (Triggers et PL/SQL).....6
5.1. Liste des Triggers Implémentés.....6
5.2. Procédures et Fonctions Stockées.....7
6. Conclusion.....7

1. Introduction

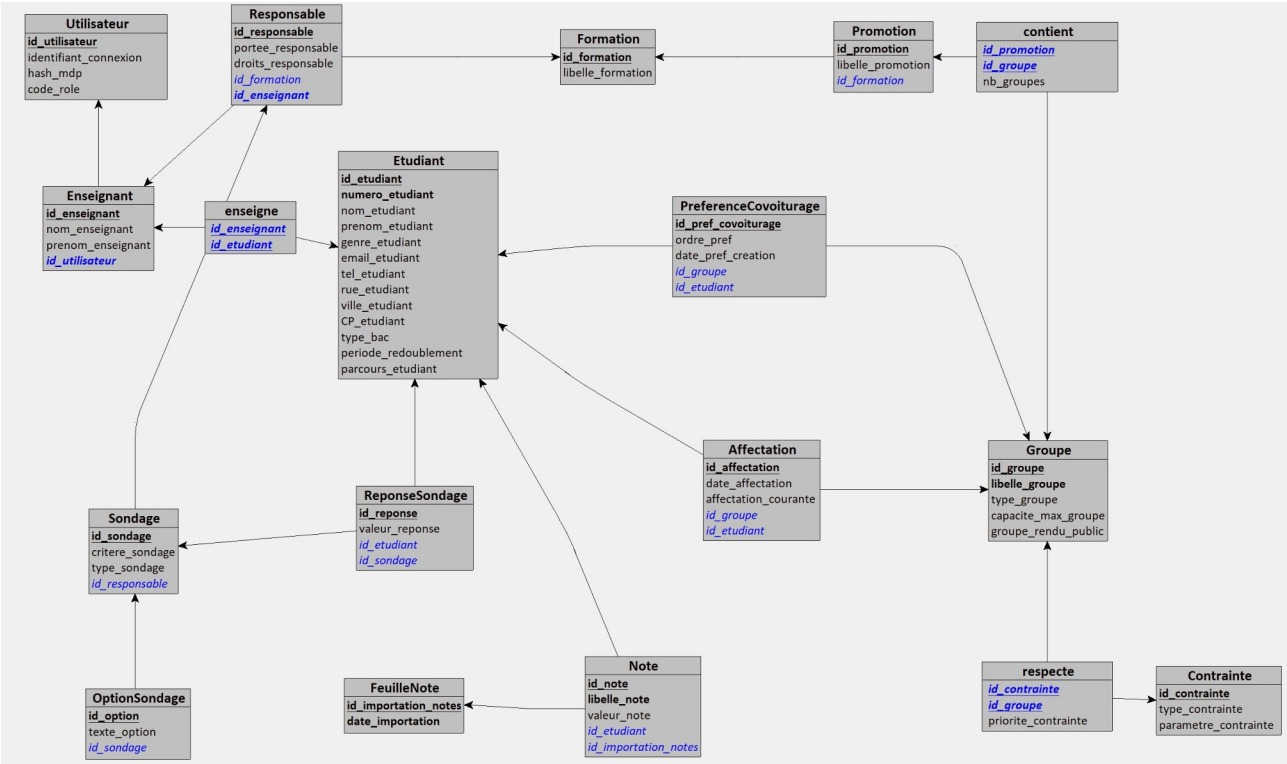
Dans le cadre de la SAE S301, ce rapport présente l'implémentation de la base de données relationnelle destinée à la gestion des affectations du département Informatique.

Ce document détaille la structure statique de la base (traduction du modèle conceptuel), les choix d'implémentation techniques, la stratégie de peuplement des données, ainsi que la logique dynamique (triggers et procédures) garantissant l'intégrité et l'automatisation des traitements.

2. Le Modèle Relationnel Commenté

Cette section présente la traduction du Modèle Entité-Association vers le Modèle Relationnel, basée sur le schéma finalisé.

2.1. Schéma Relationnel



2.2. Justification des choix d'implémentation

1. **Gestion de l'héritage (Utilisateur / Enseignant) :**

   Nous avons appliqué une stratégie par association (ou référence). La table Utilisateur centralise les informations de connexion (login et mot de passe haché) pour tous les acteurs. La table Enseignant pointe vers Utilisateur via une relation 1-1 (UNIQUE sur la clé étrangère).

   Avantage *:* Sécurité centralisée et extensibilité (facile d'ajouter un rôle "Administrateur" plus tard).

2. **Gestion de l'Historique (Table Affectation) :**

   La table Affectation est une association porteuse de propriétés. Plutôt que de supprimer les lignes lorsqu'un étudiant change de groupe, nous avons ajouté un attribut booléen affectation_courante et une date date_affectation.

   Avantage *:* Cela permet de conserver l'historique complet du parcours de l'étudiant (traçabilité) tout en identifiant facilement le groupe actuel via une clause WHERE affectation_courante = TRUE.

3. **Décomposition Promotion / Groupe (Table Contient) :**

   Une promotion (ex: BUT1) est composée de plusieurs groupes (TD1A, TP2C...). Cette relation N-N est matérialisée par la table Contient.

   Avantage *:* Souplesse pédagogique. Un même module optionnel (Groupe) pourrait théoriquement être ouvert à deux promotions différentes.

4. **Intégrité des données :**

   Des contraintes d'unicité (UNIQUE) ont été placées sur des champs clés comme numero_etudiant ou libelle_groupe pour éviter les doublons fonctionnels.

3. Implémentation Statique et Jeu de Données

Les scripts SQL fournis (Creation_tables.sql et insertions.sql) permettent de déployer la base.

3.1. Stratégie de peuplement

Volume : Plus de 20 enregistrements par table principale.
Réalisme : Nous avons simulé le département **"BUT Informatique"** complet.
Localisation : Pour coller à la réalité de l'IUT, les adresses des étudiants sont situées en Île-de-France (Essonne, Paris, Yvelines).
Cohérence : Les notes insérées correspondent bien à des matières informatiques (Java, SQL, Système) et les sondages traitent de choix pédagogiques réels (groupe projet, besoin de PC).

4. Exploitation : Vues et Requêtes

Pour répondre aux besoins fonctionnels des maquettes d'écran, nous avons implémenté **6 vues fonctionnelles** couvrant l'administration, la pédagogie et les statistiques.

4.1. Vues Pédagogiques

- **`Vue_Liste_Etudiants_Par_Groupe` :** Jointure entre `Groupe`, `Affectation` et `Etudiant`. Permet d'afficher la liste de classe instantanée en filtrant uniquement sur les affectations courantes (`TRUE`).
- **`Vue_Moyenne_Etudiant` :** Agrégation (`AVG`) des notes par étudiant. Permet de générer rapidement les bulletins ou de détecter les étudiants en difficulté.

4.2. Vues de Contrôle et Audit

- **`Vue_Alerte_Surcharge_Groupe` :** Compare le nombre d'étudiants inscrits (`COUNT`) avec la `capacite_max_groupe`. Cette vue ne retourne des lignes que si une anomalie est détectée (clause `HAVING`).
- **`Vue_Etudiants_Sans_Groupe` :** Identifie les étudiants "orphelins" qui ne sont affectés à aucun groupe actif (via une jointure gauche `LEFT JOIN` où l'affectation est `NULL`).

4.3. Vues Statistiques

- **`Vue_Resultats_Sondage` :** Comptabilise le nombre de votes par option pour chaque sondage, offrant une vision synthétique aux responsables (ex: répartition des choix de projets).

- **`Vue_Repartion_Geographique` :** Agrège les étudiants par Ville et Code Postal pour analyser la provenance des promotions.

5. Implémentation Dynamique (Triggers et PL/SQL)

Nous avons implémenté **5 déclencheurs (Triggers)** couvrant l'ensemble des opérations sensibles (Insertions et Mises à jour).

5.1. Liste des Triggers Implémentés

1. **`trigger_check_capacite` (BEFORE INSERT sur Affectation)**

   Avant d'inscrire un étudiant, le trigger compte le nombre d'inscrits actuels dans le groupe. Si ce nombre atteint ou dépasse `capacite_max_groupe`, une exception SQL est levée et l'insertion est refusée.

2. **`trigger_gestion_historique_par_type` (BEFORE INSERT sur Affectation)**

   Lorsqu'un étudiant est affecté à un nouveau groupe (ex: TD2A), le trigger désactive automatiquement son ancienne affectation **du même type** (ex: TD1C) en passant `affectation_courante` à FALSE. Cela garantit l'unicité du groupe actif par type (un seul TD, un seul TP, etc.).

3. **`trigger_check_note` (BEFORE INSERT sur Note)**

   Vérifie que toute nouvelle note insérée est strictement comprise entre 0 et 20. Rejette les valeurs aberrantes.

4. **`trigger_check_note_update` (BEFORE UPDATE sur Note)**

   Identique au précédent, mais s'applique lors de la modification d'une note existante (correction de copie).

5. **`trigger_check_unique_sondage` (BEFORE INSERT sur ReponseSondage)**

   Vérifie si l'étudiant a déjà répondu au sondage concerné. Si oui, bloque la nouvelle insertion pour garantir l'équité des votes (un étudiant = une voix).

5.2. Procédures et Fonctions Stockées

Pour simplifier l'utilisation de la base par l'application, nous avons également créé :

- **Fonction `get_places_restantes(id_groupe)` :** Retourne le nombre de places libres en temps réel.
- **Fonction `calculer_moyenne(id_etudiant)` :** Retourne la moyenne générale d'un étudiant.
- **Procédure `inscrire_etudiant` :** Encapsule la logique d'inscription complète (vérification des places via la fonction + insertion sécurisée).

6. Conclusion

Le projet rendu respecte l'ensemble des contraintes : le schéma est normalisé, les scripts sont exécutables séquentiellement sans erreur, et l'aspect dynamique sécurise les règles de gestion critiques (capacités, notes, unicité). Les scripts joints contiennent l'intégralité des sources SQL et le fichier README contient les informations nécessaires pour utiliser notre projet.